Deep Learning Aided Sensor Fusion for Drift Reduced IMU Orientation Estimation

Rahul Verma
201629064

Supervisor: Dr Zhiqiang Zhang

A project presented for the degree of
Masters of Engineering in Electronic Engineering

School of Electrical and Electronic Engineering
University of Leeds

Abstract

Inertial Measurement Units (IMUs) are widely used in a variety of applications such as Body Sensor Networks (BSNs) for orientation estimation, however the gyroscope suffers from drift due to sensor bias and noise that when integrated accumulate over time. This project investigates a deep learning-based approach which aims to mitigate gyroscopic errors which can be integrated with sensor fusion techniques to achieve more accurate orientation estimates. The proposed deep-learning architecture leverages both neural networks and a temporal history to learn complex and nonlinear error patterns in IMU data, exploring if it outperforms a standard Kalman Filter without learned corrections. The network outputs a correction for the incoming gyroscope sample and ad the measurement noise covariance dependent on the incoming acceleration and magnetometer updates. The data used in training, testing and validating the model come from simulations through MATLAB's Navigation Toolbox and from public datasets such as Berlin Robust Orientation Estimation Assessment Dataset (BROAD).

Acknowledgements

I would like to express my sincere gratitude to Professor. Zhiqiang Zhang for his continuous support and guidance throughout this project. Professor. Zhang's insights and advice during this project, in particular the quaternion algebra and an indepth explaination of filters were invaluable in shaping the direction of my work. In addition, offering his personal copy of Body Sensor Networks to aid in my understanding is highly appreciated.

I would also like to acknowledge the authors and maintainers of the publicy available datasets of BROAD and RepoIMU. Their efforts in collecting and labelling high quality IMU data and optical motion capture ground truth enabled the training, testing, and validation of the neural network. I am grateful for their contributions and cite them accordingly in this report.

Contents

List of Figures

List of Tables

1 Introduction

1.1 Inertial Measurement Units (IMUs) and their applications

IMUs are composed of multiple sensors which include a gyroscope, accelerometer, and occasionally a magnetometer. These three sensors measure the angular rate, specific force and the local magnetic field vector respectively. These measurements can be used to estimate the orientation of an object through integration of the angular rate. This data acquisition is essential for several applications such as BSNs **FIXME:** cite, robotics, and autonomous vehicles where these systems rely on high-rate orientation updates.

IMUs have emerged as a key technology due to the ability to work in a self- contained environment. In environments where external references of orientation are unavailable or unreliable, technologies such as IMUs are attractive. Filters, such as the Kalman filter (KF), allow the use of accelerometers and magnetometers to guide and correct state estimation, but these also suffer from failures like magnetic disturbances and high linear accelerations. Due to these limitations, there is significant motivation to explore data-driven methods that compensate for these conditions and errors.

WORDS: 170

1.2 Problem Statement: IMU Drift and Its Impact

IMUs have shown promise in determining the orientation of an object in motion. However, IMUs suffer from a limitation called drift. IMU drift is characterised by the accumulation of errors through the integration of the angular rate. The sources of these errors include constant bias, scale factor errors and others expanded in section 3.1.2. Errors are not exclusive to the gyroscope but also affect the accelerometer and magnetometer. Drift is also dependent on the type of IMU that is used. Lower cost/grade IMUs suffer from drift at a higher magnitude which results orientation inaccuracies much quicker compared to higher cost/grade IMUs. Errors then lead to inaccuracies in the orientation estimation of an object determined by the IMU.

Kianifar et al. explored using IMUs for orientation estimation in a clinical setting. They found that for rotation angles parallel to gravity, drift due to gyroscope bias cannot be compensated by the accelerometer.**FIXME:** cite. Even with multiple sensors, it is still challenging to find an accurate orientation estimation. Thus it is important to try and address the orientation problem by addressing gyroscopic drift.

Therefore, this project aims to address gyroscopic drift by using deep-learning methods to learn complex and non-linear nature of biases and errors.

WORDS: 211

1.3 Research Question and Hypotheses

The question this project aims to answer is: **Can Deep Learning be used to learn the drift pattern to get drift free orientation estimation?** While this is the overarching question, it can be broken down to the following:

- How effectively can deep learning learn the drift experienced?
- Can this model be generalised or will the model only apply to a single dataset?
- What are the advantages or disadvantages of using Deep Learning compared to traditional approaches?

The hypothesis that I state is that the by incorporating a deep-learning model, we will be able to see a percentage decrease in the orientation error over a period of time.

2 IMU Basics and Operational Principles

As mentioned before, IMUs consists of tri-axial gyroscope, accelerometer, and magnetometer. They measure angular rate, specific force, and the local magnetic field vector. The sensors are mounted so that it measures their components in the three orthorgonal axes x^b, y^b, z^b.

2.1 Gyroscope: Operations and Errors

The gyroscope is the main component that is used to determine the orientation of the object. It measures the angular rate in its orthorgonal axes $\omega^x, \omega^y, \omega^z$ which is used to determined the object's orientation at a discrete time. The orientation is determined through the integration of angular rate. There are multiple different ways to represent the orientation of the object, these being, Euler angles, rotation matrices, and quaternions. Euler angles give rise to the singularity problem due to the loss of one degree-of-freedom whenever two axes of the rotations are parallel **FIXME:** cite. Rotation matrices are also used, however they are less concise as they are represented as 3x3 matrix, where 6 of these elements are redundant**FIXME:** cite. Therefore in this chapter, the quaternion representation is selected due to being more computationally efficient than the others **FIXME:** cite prof paper.

2.1.1 Angular Rate Integration

Starting in continuous-time kinematics, we can define a quaternion that is represented by the angular rate, where ω is the real angular rate, $\omega_q(t)$ is the pure quaternion. Note: Both the attitude and updated quaternions should be normalised to ensure quaternions represent rotations. $q_k \leftarrow \frac{q_k}{\|q_k\|}$.

$$\omega_q(t) = [0,\ \omega_x(t),\ \omega_y(t),\ \omega_z(t)] \tag{1}$$

The orientation evolves according to**FIXME:** cite

$$\dot{q}(t) = \frac{1}{2} q(t) \otimes \omega_q(t) \tag{2}$$

where $\otimes$ denotes quaternion multiplication. The solution over $[t_0, t]$ can be written using the quaternion exponential, this assumes that the ω_q is constant over τ.

$$q(t) = q(t_0) \otimes \exp(\frac{1}{2} \int_{t_0}^{t} \omega_q(\tau)\, d\tau) \tag{3}$$

These equations show that the orientation update is determined by integrating the angular rate over time and mapping the resulting rotation into a quaternion via the exponential.

Moving to discrete-time kinematics, we can restate of defintions in terms Δt. Here the orientation evolves according to

$$\hat{q}_k = \hat{q}_{k-1} \otimes \Delta q_k \tag{4}$$

where Δq_k is

$$\Delta q_k = \exp(\frac{1}{2} \omega_{q,k} \Delta t) \tag{5}$$

This was all done by using an ideal ω_k, if were to model the angular rate from a gyroscope as **FIXME:** cite

$$\omega_{m,k} = \omega_k + b_k + n_k \tag{6}$$

Where $\omega_{m,k}$ is the measured angular rate, ω_k is the true angular rate, b_k is the bias term, and n_k is the noise term. The real quaternion orientation update is as follows.

$$\hat{q}_k = \hat{q}_{k-1} \otimes \exp(\frac{1}{2}(\omega_k + b_k + n_k)_q \Delta t) \tag{7}$$

This shows that the inclusion of the bias and noise accumulates over samples as we move forward to the next k, q_{k-1} will incorporate the previous error terms. This results in an accumulated orientation error.

2.1.2 Gyroscope Error Sources

Constant Bias

The bias of a gyroscope is the average output from the gyroscope when it is not undergoing any rotation **FIXME:** cite. This is measured in $^\circ/h$ and can be estimated by taking an average of the output. As this bias is constant, the drift it causes grows linearly with time. However, as discussed further in this section other error sources can make this difficult to determine. **FIXME:** Figure shows how constant bias changes the output.

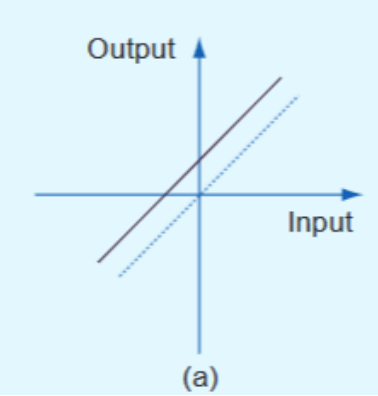

Figure 1: Bias Effect on Gyro **FIXME:** cite

White Noise / Angle Random Walk (ARW)

The gyroscope is also affected by some white noise **FIXME:** cite. This white noise sequence is zero-mean uncorrelated random variables between samples and across axes. When the gyroscope signal is integrated to obtain an angle, this white noise produces an ARW. The units of ARW is denoted by $^\circ/\sqrt{h}$, this shows that the deviation of angle error grows proportionally to $\sqrt{t}$. As ARW is due to random variables, it is classified as the 1σ of the orientation error. Analog devices shows this in the **FIXME:** figure, where the ARW is $0.17^\circ/\sqrt{h}$.

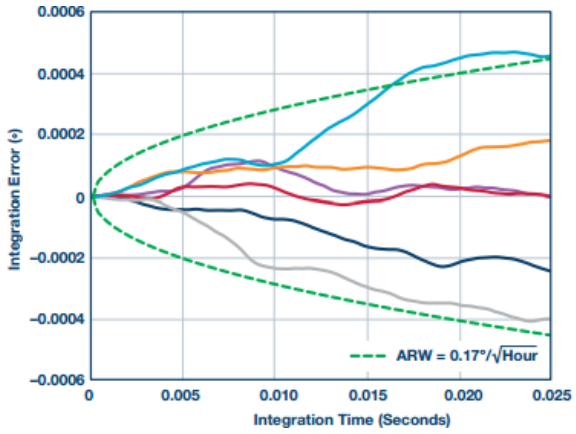

Figure 2: Effect of ARW on Gyro Output **FIXME:** cite

Flicker Noise / Bias Stability

The gyroscope suffers from flicker noise in electronics **FIXME:** cite Flicker noise is $1/f$, due to this the effects are observed at lower frequenices while, higher frequenices the noise is dominated by white noise.

Temperature

Temperature fluctuations due to changes in the environment and sensor heating induce a movement in the bias, this relationship is also highly non-linear **FIXME:** cite. Therefore, it can be very difficult to model and subsequently subtract from the gyroscope measurements compared to a constant bias.

2.2 Accelerometer: Operations and Errors

The accelerometer is another component used in determining an orientation estimate of an object. It measures the specific force along its orthogonal axes a^x, a^y, a^z. These measurements can be rotated in the navigation frame, gravity vector is compensated, and through integration of the accelerometer, a velocity and subsequently the position can be derived for an object. However, the focus of this project is to achieve drift-free orientation, and hence this will not be covered.

The main purpose of the accelerometer is to offer a gravity reference to the system such that, an accelerometer is capable of providing drift-free inclination estimates **FIXME:** cite. However, the accelerometer, as stated above, does not only measure a gravity reference but also any linear accelerations that the object also experiences. Under conditions of high accelerations, the reference is unreliable and is only reliable for static or slowly moving objects **FIXME:** cite. Additionally, the accelerometer cannot detect rotations about the gravity vector, so it cannot provide a yaw/heading of the object. However, like the gyroscope, it also suffers from a variety of errors which causes instability in the output.

2.2.1 Accelerometer Error Effects

Accelerometers suffer from a very similar sources of errors to the gyroscope (section 3.1.2). Therefore, in this section we will discuss the implications of these errors for our orientation estimates.

Constant Bias

The effect of constant bias is that it causes an offset in the roll/pitch of the system. Unlike constant bias in the gyroscope, it does not grow linearly with time but is a constant offset irrespective of time elapsed.

High Linear Acceleration

As discussed above, the largest problem is high linear accelerations which corrupts the gravity reference. This is a dominant issue in high dynamic motions as the accelerometer will not be able to correct the orientation estimation through fusion **FIXME:** section, especially if gyroscopic drift is not addressed.

A potential plan to address this will be discussed in **FIXME:** section

2.3 Magnetometer: Operations and Errors

Magnetometers measure the local magnetic field along its orthogonal axes m^x, m^y, m^z. While the accelerometer can help in determining the inclination (roll/pitch), of an object, it cannot observe any rotations along the gravity vector. This is where a magnetometer can be used to determine the heading (yaw) of an object. Sensor fusion/filtering algorithms are dependent on sensing the local magnetic flied to eliminate drift in the azimuth (yaw) portion of orientation estimates **FIXME:** cite. It does this by assuming that the measured local field vector provides a reference relative to Earth's field, however changes in the local field due to other field sources, can distort this measurement and gives rise to erroneous heading corrections.

2.3.1 Magnetometer Error Sources and Effects

Hard Iron/ Soft Iron

Hard Iron is a constant additive magnetic field applied to the sensor due to the interaction of magnetic fields **FIXME:** cite 2. An example of this can be a permanent magnet near the magnetometer interacting

with Earth's field. This interaction between these sources leads to a constant vector being applied to the measurement. Whilst this constant vector is easy to subtract in theory, it is very difficult to measure the Earth's field ,in the local frame, without any interference of any kind.

Soft Iron are the effects generated by the interaction of an external magnetic field on any ferromagnetic materials near the sensor **FIXME:** cite 2. The resulting magnetic field depends on the magnetic field vector with respect to the orientation soft iron material **FIXME:** cite 3. This effect is difficult to determine as if the orientation of the sensor changes, then the orientation with respect to the soft iron also changes which distorts the field in a different way.

Noise

As previously stated, in each sensor there is some noise that will either arise from flicker noise or noise at higher frequencies which are modelled as white noise in nature. However, like the accelerometer, we do not need to do any type of integration which would lead to this noise giving an unbounded drift. Therefore, even though it is an error source any hard/soft iron effects would dominate any effect that noise would have on the system.

3 Deep Learning Architecture

3.1 Aims of the Deep Learning Model

Referring back to section 1.2, the overall problem is to implement a model that can output gyroscopic corrections $\Delta\boldsymbol{\omega}_t = [\Delta\omega_x, \Delta\omega_y, \Delta\omega_z]^\top$ that when subtracted from our measured angular rate, and fused with other sensors such as an accelerometer and magnetometer leads to an orientation estimate, that is more accurate to the ground truth orientation of an object compared with only sensor fusion.

The motivation for learning $\Delta\boldsymbol{\omega}_t$ comes from the fact that while sensor fusion techniques of downweighting or skipping of the accelerometer and magnetometer when they are unreliable , drift cannot be mitigated due to the accumulation of gyroscopic bias and noise **FIXME:** cite, cite2.

3.2 Model Selection

3.2.1 Recurrent Neural Networks (RNNs)

The first type of model that was considered is a RNN. RNNs stem from an idea of recurrence, that is that the output depends on some input at time t and an internal state of the model that holds information about previous outputs of the model. The general idea can be described by the equation

$$\hat{y} = f_{model}(x_t, h_{t-1}) \tag{8}$$

where $\hat{y}$ is the output, f_{model}, is a function that describes the model, x_t is the input at time t, and h_{t-1} is a vector state that describes the internal state of the model at the previous time step. Expanding on this the state of the model is updated as follows

$$h_t = f_w(x_t, h_{t-1}) \tag{9}$$

where f_w is a function with weights w. This expands to the equations

$$\hat{y} = W_{hy}^T h_t \tag{10}$$

$$h_t = \phi(W_{hh}^T h_{t-1} + W_{xh}^T x_t) \tag{11}$$

where $W_{hh}^T, W_{hy}^T, W_{xh}^T$ are a set of weight matrices that are reused for every time step, and ϕ represents an arbitrary activation function.

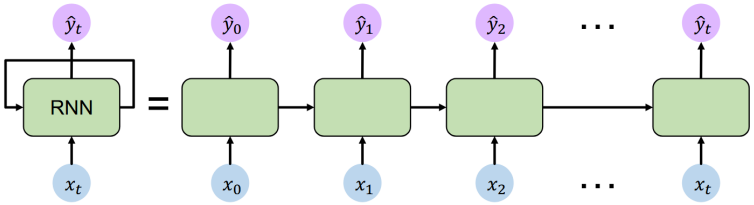

Figure 3: RNN structure **FIXME:** cite3

Figure 3 shows the unrolled flow of the RNN, where the arrows between each block represents the passing of h_t, from one input to the next. The point of the recurrence state h_t, is to show how the output of the model depends on context from previous inputs plus h_t. This is especially relevant in our case, as described in section 3.1.1, the accumulation of gyroscopic errors comes from the integration of the angular rate. As these errors accumulate, the current error depends on the history of the gyroscope measurements.

There are many problems with RNNs however, the biggest being the problem of exploding/vanishing gradients. RNNs are trained via backpropagation through time, which propagates gradients across timesteps.

For long sequences, computing the gradient with respect to h_0 involes many factors of repeated multiplication of Jacobians across time. This can lead to many values of the computation being greater than 1, which explodes gradients leading to unstable training. In the case, where many values are less than 1, this causes vanishing gradients where learning long-range dependencies becomes difficult. Additionally, as the hidden state h_t is propagated sequentially through time, when performing backpropagation, training cannot be parallelised over timesteps. This results in a high training times compared to convolutional models.

Updated versions of the RNN have been proposed to solve the problem of vanishing and exploding gradients. These being Long Short Term Memory (LSTM) and Gated Recurrent Units (GRUs). These methods exploit the use of a cell state that does not need to do perform repeated multiplication of Jacobians, however they are still subject to long training times **FIXME:** cite and the gradient problem is not fully solved but mitigated **FIXME:** cite, cite.

3.2.2 Convolutional Neural Networks (CNNs)

The next type of model considered was the CNN, even though typically it is used in computer vision tasks; it has seen wide use in research relating to drift reduction **FIXME:** cite, cite, cite. This model is based on the idea of convolving input data with a learnable filter to extract local features in the input data. The filter can be thought of like a window that slides over the data, and each filter has a defined set of weights in which it multiplies with input data. This convolution operation can be described by the equation below:

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t + k - \lfloor K/2 \rfloor] \tag{12}$$

$$y_f[t] = \phi(z_f[t]) \tag{13}$$

where $y_f[t]$, is the output of filter f, b_f is a bias, C_{in} is the input channels (for our case C_{in} would be 3 for $\omega_x, \omega_y, \omega_z$), K is the kernel/filter width, $w_{f,c}$ is the weights of the filter for a specific channel, x_c is the input data of that channel, and ϕ is an arbitrary non-linear activation function (typically ReLU).

For example if $K = 3$, the convolution will occur of the timesteps $x[t-1], x[t], x[t+1]$, this is how the convolution extracts local features in the input data.

Typically, these CNNs are made up of layers, where in each layer the number of filters grow e.g. $L1 = 16, L2 = 32, L3 = 64$ etc. These additional filters in later layers allow the model to learn more complex feature sets based on the previous layers' filter. Zeiler et al. **FIXME:** cite shows this in their figure 2 where their convolutional network is used for image classification.

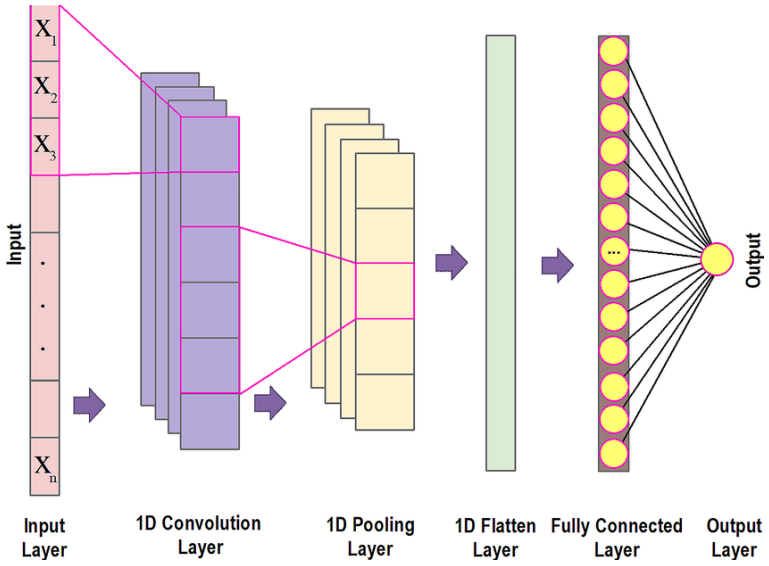

Figure 4: 1D CNN structure **FIXME:** cite

CNNs are more attractive than RNNs due to its ability to be efficiently trained by using parallelisation**FIXME:** cite. As stated before, RNNs cannot exploit this, but CNNs can. This leads to the ability to test multiple different iterations of a model where parameters and loss functions may be changed to explore what effects they may have **FIXME:** see section. CNNs also do not suffer from vanishing//exploding gradients as much and are seen to be more stable during training**FIXME:** cite.

However, unlike RNNs, baseline CNNs cannot produce outputs due to significant temporal information. It can extract local features and in 1D convolutions it has some ability to traverse backwards through time but not to any significant degree. As seen in section 3.1.1, the accumulation of drift is due to the integration of the angular rate as we move forward in time.

3.2.3 Temporal Convolutional Networks (TCNs)

TCNs are an extension of the baseline CNN and include two main differences in their implementation as shown by Bai et al.**FIXME:** cite. These differences are the use of dilations that grow the receptive field allowing the network to look back further in time and learn long-term information. Furthermore, causal convolutions meaning, that the convolution operation cannot look forward in time. To explain causality we can look back at equation 12 and modify this so

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t-k] \tag{14}$$

$$y_f[t] = \phi(z_f[t]) \tag{15}$$

The difference is that the k is subtracted from $x[t]$. The causality is enforced here as at time t, we cannot access future information unless the model is to be used after data collection and not in a real time setting.

Dilations allow the model to look further back in time without the need of increasing parameters for the same K and C_{in} used in convolutions. This is best represented in the figure 5.

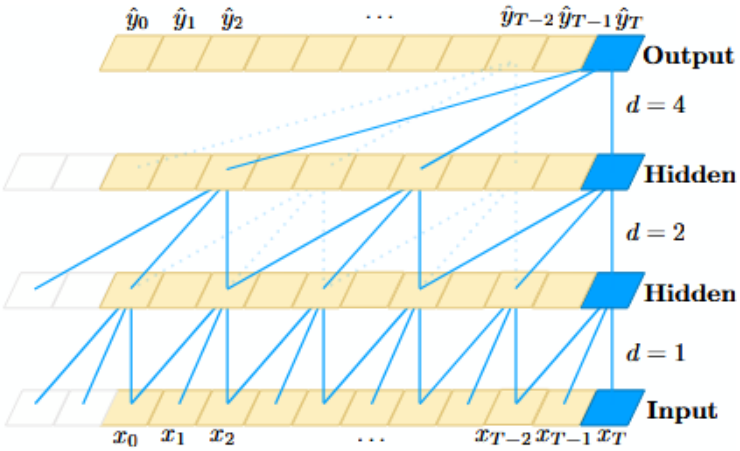

Figure 5: CNN structure with dilations **FIXME:** cite

As we can see in the diagram, as the dilations increase layer by layer the output to the next layer looks further back into the history of $x[t]$. This can be modelled by the equation

$$z_f[t] = b_f + \sum_{c=1}^{C_{in}} \sum_{k=0}^{K-1} w_{f,c}[k] x_c[t - dk] \tag{16}$$

where d is the dilation factor of the layer.

3.2.4 Summary and Choice

In summary, the choice has been to select the TCN. This is because it has the ability to look back in history and model long-term dependencies between data, something baseline CNN cannot do. Additionally, like a CNN it is quicker and more stable in training unlike an RNN. However, the TCN does have its problems, these being extra data storage during its operation and domain transfer. Unlike RNNs which only have store $h(t)$, TCNs needs to take in a raw sequence upto its effective history length **FIXME:** cite. Domain transfer refers to transferring a model that has been trained in one domain (i.e. small receptive field) and applied to another domain (i.e. large receptive field), where the TCN may perform poorly due to this change in domain.

3.3 TCN Design

The main design of the TCN comes from Bai et al. **FIXME:** cite. Figure 6 demonstrates the architecture of the residual block that has been implemented.

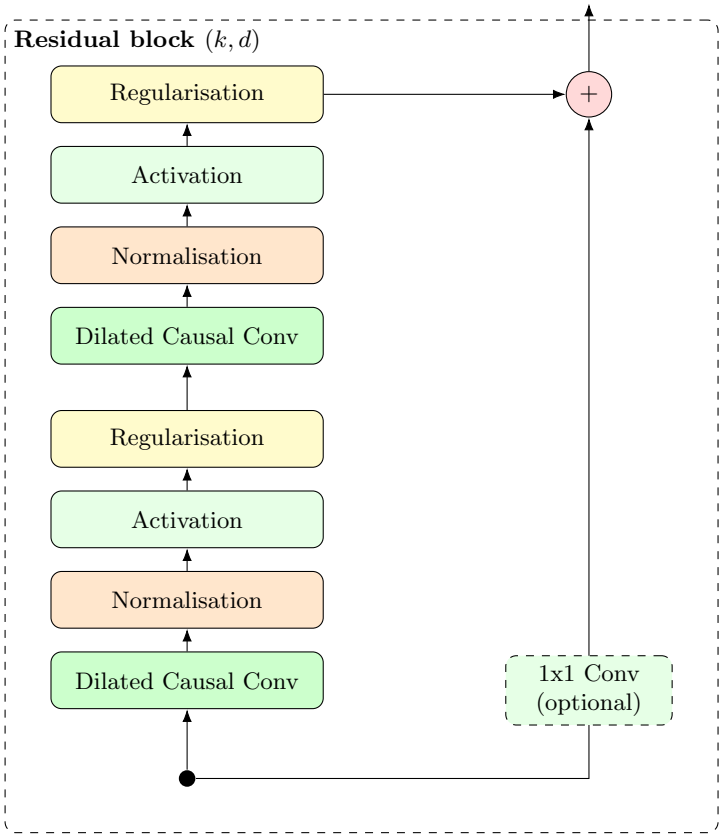

Figure 6: Structure of residual block

The dialted causal convolution is the same as discussed in section 3.2.3.

3.3.1 Normalisation

Normalisation is a technique used in deep learning to ensure training stability, while also reducing training times by allowing the use of higher learning rates. The aim of normalisation is to address internal covariate shift, the change in the distrubution of each layer's inputs during training, as parameters of the previous layers changes **FIXME:** cite. This can be done by the following techniques.

Method	Normalises	Stats over	Pros / Cons
BatchNorm	Activations (per channel)	Batch (and spatial/time dims)	+ Fast and adds some regularisation – small batch/streaming issues
LayerNorm	Activations (per token/sample)	Features within a sample	+ Batch-size independent – sometimes worse performance in CNNs
GroupNorm	Activations (channel groups)	Groups of channels within a sample	+ Works well with small batch – Number of groups need to be selected
WeightNorm	Weights (reparam)	Weight vector norm (no batch stats)	+ Simple no batches – less regularisation than BatchNorm

Table 1: Common normalisation methods.

From the table above, in training BatchNorm, GroupNorm, and WeightNorm will be tested to see which technique yields the best model accuracy to model training stability. The reason why these have been selected is that BatchNorm has seen wide use in CNN models for drift reduction **FIXME:** cite brossard, cite chen. WeightNorm as the original TCN implementation uses it to avoid errors with small batch sizes **FIXME:** cite. GroupNorm as it is a middle ground where it works well with small batch sizes compared to BatchNorm but it may introduce more complexity with groups needing to be selected and as such will be tested last to see if the other techniques give an acceptable tradeoff.

3.3.2 Activation Functions

Activation fucntions are functions that are used in deep learning to add non-linearity to our output. This is because seen in equations 10,12,14, the relationship between weights and input data is all linear. If the relationship between input and output were linear there would be no need for a deep learning model as these relationships could be modelled easily. The table below demonstrates common activation functions used.

Activation	Definition	Output range	When used	Pros / Cons
ReLU	$\text{ReLU}(x) = \max(0, x)$	$[0, \infty)$	CNN default baseline	+ Fast activations – ReLU can result in 0 output (dying ReLU)
Leaky ReLU	$\max(\alpha x, x)$, $\alpha \in (0, 1)$	$(-\infty, \infty)$	When ReLU units die	+ Reduces dying ReLU – extra hyperparameter α
GELU	$x\,\Phi(x)$ (Gaussian CDF)	$(-\infty, \infty)$	Transformers, deep networks	+ Smooth and good performance – slower than ReLU
Swish	$x\,\sigma(\beta x)$	$(-\infty, \infty)$	Transformers/CNNs	+ Smooth, often improves accuracy – slower than ReLU

Table 2: Common activation functions.

From the table above, the motivation for ReLU is that it is commonly used as the baseline for CNNs and is also used by Bai et al. in their TCN implementation. However, they can be prone to overfitting **FIXME:** brossard cite or dying ReLU can occur. If ReLU is prone to dying ReLU, then leaky ReLU will be used and tested against the other functions. GELU and Swish are known to be smooth implementations of ReLU, where it is able to take values where $x < 0$ and can have slight performance improvements **FIXME:** cite activations but are computationally more expensive and performance is model dependent. For implementation, ReLU will be considered first, however if problems arise the other activation functions will be used.

3.3.3 Regularisation

Regularisation is a technique used in deep learning models to help models avoid overfitting to training data and allows the model to be more generalisable to unseen data. The technique that will be implemented first is dropout, where dropout randomly zeroes out all activations within the model for an input in training data. If the model is still prone to overfitting, more advanced regularisation techniques may be employed like weight decay which penalises large weights within kernels to allow for smoother functions.

3.3.4 Residual Connections/1x1 Convolution

The right branch (figure 6) is essential in training very deep neural networks where networks may not be able to converge due to exploding/vanishing gradients. This branch is called a residual connection, where the equation below shows its operation

$$y = \mathcal{F}(x, W_i) + Wx \tag{17}$$

where y is the output, $\mathcal{F}(x, W_i)$ is the residual mapping to be learned, W, is a projection applied to x (1x1 convolution), and x is our input**FIXME:** cite residual.

Residual connections adds the input of a block to the output of the block, so when learning the model learns the relationship of $\mathcal{F}(x, W_i) = y - Wx$, which is easier to learn than the full mapping directly. The 1x1 convolution is included as the model goes deeper, the number of channels increases to capture more complex features as each channel has their own set of filter/kernels. The addition between $\mathcal{F}(x, W_i)$ and x must be of two tensors of the same size so W allows the addition to be possible.

3.3.5 Loss Functions

4 Conclusion and Next Steps

4.1 Semester 2 Aims

4.2 Summary of Progress

During Semester One, the project progressed from initial understanding of the challenges faced in orientation estimation, to exploration and designing of deep learning models. The majority of the work conducted had been research through the use of academic literature and online lectures. With foundational knowledge gained in all areas of the project, it has ensured that Semester Two can be focused on the completion deep learning model to address the questions posed in section 1.3.

4.3 Challenges

The primary challenge encountered during this semester stemmed from having no prior knowledge of IMU operations and deep learning. This led to extensive research needed in order for the completion of the project, which slowed down the implementation of the model. However, now having concluded my research I have been able to make informed decisions behind the selection of model, loss function design, optimisation, and measurements that accurately assess the performance of the model.

4.4 Skills Developed

The main skill developed has been learning how to assess and evaluate literature and ascertain the correct conclusions to be able to be applied to this project. My ability to use the PyTorch library and Python skills have developed to a significant degree in which modelling, measurements, and analysis will be conducted within Python. In addition, my technical writing capabilities have also progressed especially compared to my third year project.